

Relational Database Design

[Outline](#)

[Features of Good Relational Designs](#)

[Decomposition](#)

[Lossless Decomposition](#)

[Normalization Theory](#)

[Functional Dependencies](#)

[Closure of a Set of Functional Dependencies](#)

[Keys and Functional Dependencies](#)

[Use of Functional Dependencies](#)

[Trivial Functional Dependencies](#)

[Dependency Preservation](#)

[Normal Forms](#)

[First Normal Form](#)

[Second Normal Form](#)

[Third Normal Form](#)

[Boyce-Codd Normal Form \(BCNF\)](#)

[Decomposing a Schema into BCNF](#)

[Comparison of BCNF and 3NF](#)

[Goals of Normalization](#)

[How Good is BCNF?](#)

[Functional-Dependency Theory](#)

[Closure of a Set of Functional Dependencies](#)

[Procedure for Computing Closure of F](#)

[Closure of Attribute Sets](#)

[Uses of Attribute Closure](#)

[Canonical Cover](#)

[Extraneous Attributes](#)

[Left-hand side](#)

[Right-hand side](#)

[Canonical Cover](#)

[Dependency Preservation](#)

[Testing for Dependency Preservation](#)

[Algorithm for Decomposition using FDs](#)

[Testing for BCNF](#)

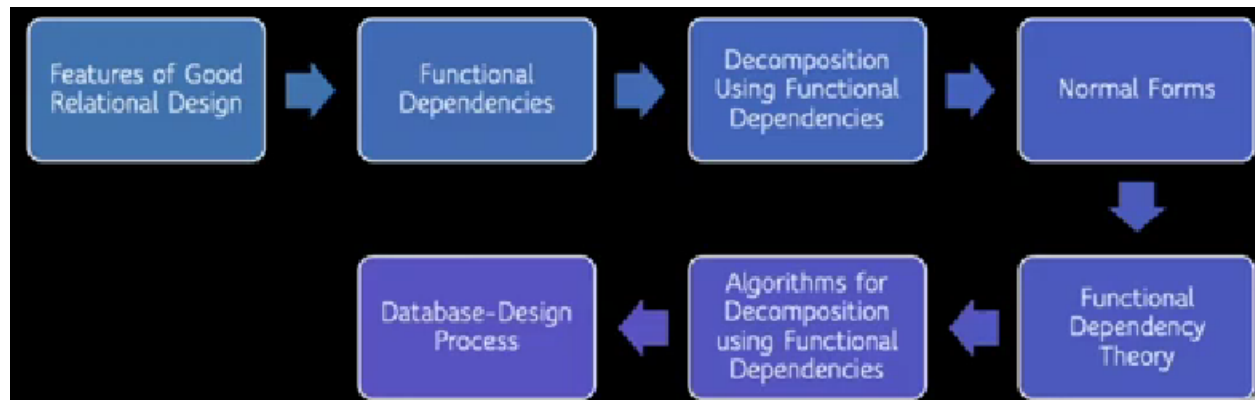
[Testing Decomposition for BCNF](#)

[BCNF Decomposition Algorithm](#)

[Testing for 3NF](#)

[3NF Decomposition Algorithm](#)

Outline



Features of Good Relational Designs

Desain yang buruk akan menyebabkan perulangan informasi (waste of space, you have to make sure the data is consistent every time you update). ketidakmampuan merepresentasikan nilai tertentu (sehingga diberikan nilai null).

Tujuan perancangan basis data relasional:

- menghindari data redundan
- memastikan hubungan antar atribut dapat direpresentasikan dalam basis data
- memfasilitasi pengecekan pada proses update untuk menghindari pelanggaran pada integrity constraints.

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Pada relasi di atas, ada perulangan informasi. Contohnya di dept_name physics di gedung watson dengan budget 70000 itu berulang 2x. Anomali: tidak bisa insert department baru

tanpa menambahkan instructor pada department baru tersebut, jika ingin menghapus seorang instructor (misalnya 98345), itu akan ikut menghapus department electrical engineering.

Decomposition

Dilakukan untuk menghindari duplikasi informasi. Berikut adalah dekomposisi relasi sebelumnya

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

ID	name	salary	dept_name
22222	Einstein	95000	Physics
12121	Wu	90000	Finance
32343	El Said	60000	History
45565	Katz	75000	Comp. Sci.
98345	Kim	80000	Elec. Eng.
76766	Crick	72000	Biology
10101	Srinivasan	65000	Comp. Sci.
58583	Califieri	62000	History
83821	Brandt	92000	Comp. Sci.
15151	Mozart	40000	Music
33456	Gold	87000	Physics
76543	Singh	80000	Finance

dept_name	building	budget
Physics	Watson	70000
Finance	Painter	120000
History	Painter	50000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Biology	Watson	90000
Music	Packard	80000

©Silberschatz et.al. (2019)
[modified by Tim Pengajar IF2140 Semester 1 2020/2021]

Not all decompositions are good. Contohnya adalah sebagai berikut:

employee(ID, name, street, city, salary)

ID	name	street	city	salary
..				
57766	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000
..				

lossy decomposition

employee1 $\bowtie$ *employee2*

ID	name	street	city	salary
..				
57766	Kim	Main	Perryridge	75000
57766	Kim	North	Hampton	67000
98776	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000
..				

Lossless Decomposition

- Lossless decomposition adalah ketika dinatural-joinkan ia akan menghasilkan relasi sebelum dekomposisi
- Let R be a relation schema and let R_1 and R_2 form a decomposition of R. That is $R = R_1 \cup R_2$
- A lossless decomposition if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$. Formally, $\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$
- A decomposition is lossy if: $r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$
 → artinya, saat dijoin akan memberikan tuple-tuple lebih banyak dari tuple pada r awal.
- A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+ . $R_1 \cap R_2 \rightarrow R_1$; $R_1 \cap R_2 \rightarrow R_2$

$R(A,B,C)$

A	B	C
α	1	A
β	2	B

$R1(A,B) = \Pi_{A,B}(r)$

A	B
α	1
β	2

$R2(B,C) = \Pi_{B,C}(r)$

B	C
1	A
2	B

$R1 \bowtie R2$

A	B	C
α	1	A
β	2	B

Normalization Theory

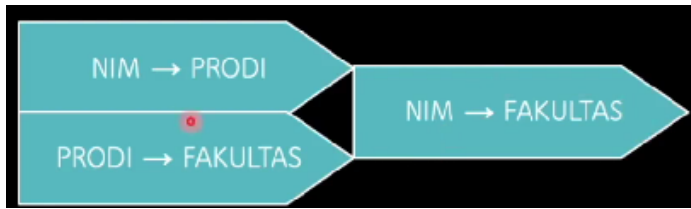
1. Decide whether a particular relation R is in "good" form
2. In the case that a relation R is not in "good" form, decompose it into set of relations $\{R_1, R_2, \dots, R_n\}$ such that each relation is in good form and the decomposition is lossless.
3. Our theory is based on
 - a. Functional dependencies $\rightarrow$ constraint yang membatasi kemungkinan munculnya suatu pasangan dalam sebuah relasi (?)
 - b. Multivalued dependencies

Functional Dependencies

- There are usually a variety of constraints on the data in the real world
For example, some of the constraints that are expected to hold in a university database are:
 - students and instructors are uniquely identified by their ID
 - each student and instructor has only one name
 - each instructor and student is (primarily) associated with only one department
 - each department has only one value for its budget, and only one associated building.
- Legal instance of the relation is an instance of a relation that satisfies all such real-world constraints
- A legal instance of a database is one where all the relation instances are legal instances
- Constraints on the set of legal relations
- the value of a certain set of attributes can be uniquely determined by the value for another set of attributes
- A functional dependency is a generalization of the notion of a key
- Let R be a relation schema. $\alpha \subseteq R$ and $\beta \subseteq R$
- The functional dependency $\alpha \rightarrow \beta$ holds on R iff for any legal relations r(R), whenever any two tuple of r agree on the attributes α , they also agree on the attributes β
 - That is, $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$

Closure of a Set of Functional Dependencies

- Given a set F of functional dependencies, there are certain other functional dependencies that are logically implied by F .



- The set of all functional dependencies logically implied by F is the **closure** of F (F^+).

Keys and Functional Dependencies

- K is a superkey for relation schema R iff $K \rightarrow R$.
- K is a candidate key of R iff $K \rightarrow R$, and for no $\alpha \subset K$, $\alpha \rightarrow R$ (tidak ada subset dari K yang berfungsi sebagai superkey)
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys.

Use of Functional Dependencies

- To test relations to see if they are legal under a given set of functional dependencies. If a relation r is legal under a set F of functional dependencies, we say that r satisfies F .
- To specify constraints on the set of legal relations. We say that F holds on R if all legal relations on R satisfy the set of functional dependencies F .
- Note: a specific instance of a relation schema may satisfy a functional dependency even if the functional dependency doesn't hold on all legal instances.

Trivial Functional Dependencies

- A functional dependency is trivial if it is satisfied by all instances of a relation
Example: $ID, name \rightarrow ID$; $name \rightarrow name$
- In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

Dependency Preservation

- If testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low
- When decomposing a relation it is possible that it is no longer possible to do the testing without having to perform a Cartesian product.

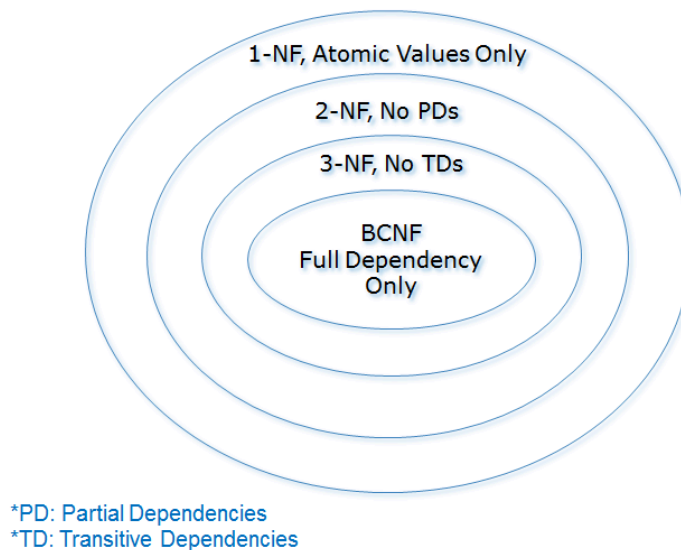
- A decomposition that makes it computationally hard to enforce functional dependency is said to be NOT dependency preserving

- Consider a schema:
 $dept_advisor(s_ID, i_ID, department_name)$
- With function dependencies:
 $i_ID \rightarrow dept_name$
 $s_ID, dept_name \rightarrow i_ID$ ❌
- In the above design we are forced to repeat the department name once for each time an instructor participates in a *dept_advisor* relationship.

s_ID	i_ID	dept_name
Clark	Srinivasan	Comp. Sci.
Clark	Wu	Info. Syst.
Bruce	Wu	Info. Syst.
Diana	Katz	Comp. Sci.
Peter	Srinivasan	Comp. Sci.
Peter	Kim	Elec. Eng..

Pada contoh di atas, dekomposisi tidak mungkin dilakukan agar dependency preserving karena pengecekan $s_ID, dept_name \rightarrow i_ID$ hanya dapat dilakukan melalui join.

Normal Forms



First Normal Form

- A relation schema R is in **first normal form (1NF)** if the domains of all attributes of R are atomic

Second Normal Form

- A relation schema R is in **second normal form (2NF)** if each attribute A in R meets one of the following criteria:
 - It appears in a candidate key
 - it is not partially dependent on a candidate key
- For example: StudentCourse(sID, sName, cID, cName, Grade)
 - $F = \{sID \rightarrow sName, cID \rightarrow cName, sID \ cID \rightarrow Grade\}$
→ sID adalah bagian dari candidate key. sName bukan candidate key dan dia hanya bergantung pada sID. Sehingga ketergantungan sName ke candidate key parsial → masih terdapat partial dependency, bukan 2NF.

Third Normal Form

- 3NF is a minimal relaxation of BCNF to ensure dependency preservation
- A relation schema R is in **third normal form (3NF)** if for all $\alpha \rightarrow \beta$ in F^+ at least one of the following holds:
 - $\alpha \rightarrow \beta$ is trivial (i.e. $\beta \in \alpha$)
 - α is a superkey for R
 - Each attribute in $\beta - \alpha$ is contained in a candidate key for R
- Redundancy in 3NF

```
dept_advisor(s_ID, i_ID, dept_name)
F = { i_ID → dept_name,
      s_ID, dept_name → i_ID }
```

- Two candidate keys = {s_ID, dept_name}, {s_ID, i_ID}
- dept_advisor is **not in BCNF**
- dept_advisor, however, is in **3NF**
 - s_ID, dept_name is a superkey
 - i_ID is NOT a superkey, but dept_name is contained in a candidate key

Consider schema dept_advisor(s_ID, i_ID, dept_name) with functional dependencies

$$F = \{ i_ID \rightarrow dept_name, s_ID, dept_name \rightarrow i_ID \}$$

which is in 3NF.

s_ID	i_ID	dept_name
Clark	Srinivasan	Comp. Sci.
Clark	Wu	Info. Syst.
Bruce	Wu	Info. Syst.
Diana	Katz	Comp. Sci.
Peter	Srinivasan	Comp. Sci.
Peter	Kim	Elec. Eng..

What is wrong with the table?

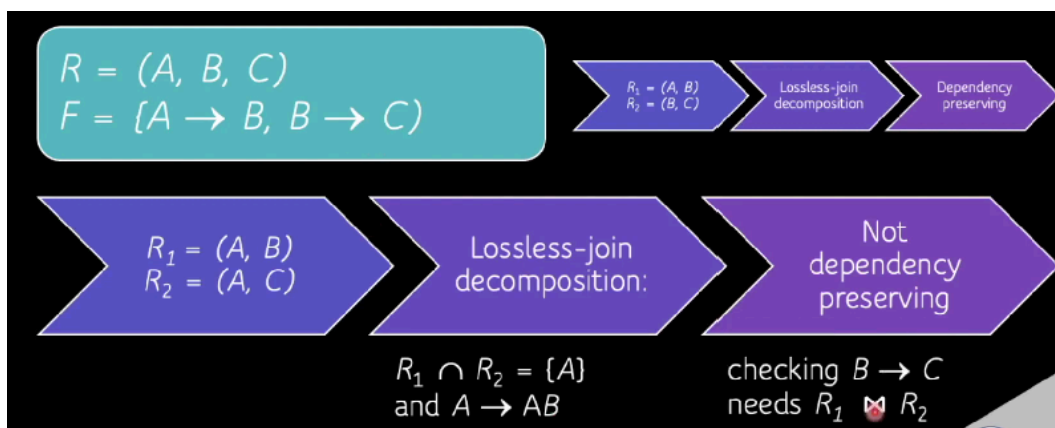
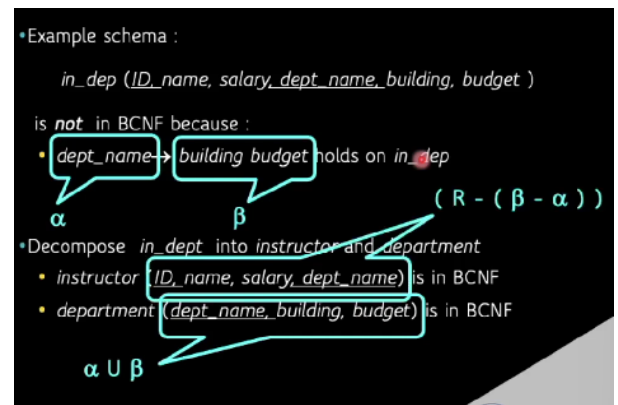
- repetition of information
- need to use null values (i.e. to represent the relationship Einstein, Physics where there is no corresponding value for s_ID)

Boyce-Codd Normal Form (BCNF)

- A relation schema R is in BCNF with respect to a set F of functional dependencies if for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:
 - $\alpha \rightarrow \beta$ is trivial (i.e. $\beta \subseteq \alpha$)
 - α is a superkey of R
- For example: $\text{in_dep}(\underline{\text{ID}}, \text{name}, \text{salary}, \text{dept_name}, \text{building}, \text{budget})$ is **not in BCNF** because:
 - $\text{dept_name} \rightarrow \text{building budget}$ (FD ini tidak trivial, dan dept_name bukan merupakan superkey)
- We can decompose in_dept into instructor and department
 - $\text{instructor}(\underline{\text{ID}}, \text{name}, \text{salary}, \text{dept_name})$ is in BCNF
 $\rightarrow$ karena satu instructor satu ID hanya memiliki satu name, satu salary, dan satu dept_name. sehingga $\text{ID} \rightarrow \text{name salary dept_name}$ dan LHS atau ID merupakan atribut superkey.
 - $\text{department}(\underline{\text{dept_name}}, \text{building}, \text{budget})$ is in BCNF
 $\rightarrow$ karena $\text{dept_name} \rightarrow \text{building budget}$ dan dept_name merupakan superkey.

Decomposing a Schema into BCNF

- Let:
 - R be a schema that is not in BCNF
 - $\alpha \rightarrow \beta$ be the FD that causes a violation
- We decompose R into:
 - $(\alpha \cup \beta)$
 - $(R - (\beta - \alpha))$
- Contoh lain:



- BCNF and Dependency Preservation:
 - it isn't always possible to achieve both BCNF and dependency preservation

Comparison of BCNF and 3NF

Advantages to 3NF over BCNF	Disadvantages to 3NF
It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation	We may have to use null values to represent some of the possible meaningful relationships among data items
	There is the problem of repetition of information

Goals of Normalization

- Let R be a relation scheme with a set F of functional dependencies
- Decide whether a relation scheme R is in "good" form
- In the case that R is not in "good" form, need to decompose it into {R₁, R₂, ..., R_n} such that:
 - each relation scheme is in good form
 - the decomposition is a lossless decomposition
 - preferably, the decomposition should be dependency preserving

How Good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation inst_info(ID, child_name, phone) where an instructor may have more than one phone and can have multiple children

ID	child_name	phone
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	William	512-555-4321

- Insertion anomalies: if we add a phone 981-992-3443 to 99999, we need to add two tuples (99999, David, 981-992-3443) and (99999, William, 981-992-3443)

- It is better to decompose inst_info into:

• *inst_child*:

ID	child_name
99999	David
99999	William

• *inst_phone*:

ID	phone
99999	512-555-1234
99999	512-555-4321

This suggests the need for higher normal forms, such as 4NF

Functional-Dependency Theory

Closure of a Set of Functional Dependencies

We can compute F^+ , the closure of F , by repeatedly applying Armstrong's Axioms:

- Reflexive** rule: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
- Augmentation** rule: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
- Transitivity** rule: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

Additional rules:

- Union** rule: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- Decomposition** rule: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- Pseudotransitivity** rule: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Procedure for Computing Closure of F

To compute the closure of a set of functional dependencies F :

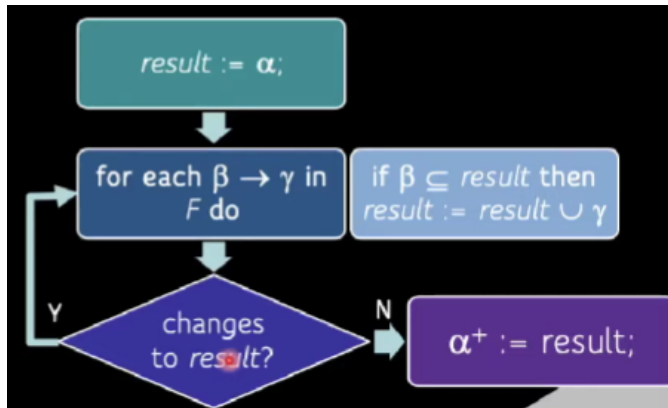
```

 $F^+ = F$ 
repeat
  for each functional dependency  $f$  in  $F^+$ 
    apply reflexivity and augmentation rules on  $f$ 
    add the resulting functional dependencies to  $F^+$ 
  for each pair of functional dependencies  $f_1$  and  $f_2$  in  $F^+$ 
    if  $f_1$  and  $f_2$  can be combined using transitivity
      then add the resulting functional dependency to  $F^+$ 
until  $F^+$  does not change any further

```

Closure of Attribute Sets

- Given a set of attributes a , define the closure of a under F (denoted by a^+) as the set of attributes that are functionally determined by a under F .
- Algorithm to compute α^+ :



- Application example of the above algorithm:

$R = (A, B, C, G, H, I)$

$F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

$(AG)^+$?

result = AG

1. result = ABCG ($A \rightarrow B, A \rightarrow C$ and $A \subseteq AB$)

2. result = ABCGHI ($CG \rightarrow H, CG \rightarrow I$ and $CG \subseteq ABCG$)

Is AG a candidate key?

1. Is AG a superkey? yes.

a) Does $AG \rightarrow R$? == is $R \subseteq (AG)^+$

2. Is any subset of AG a superkey?

a) Does $A \rightarrow R$? == is $R \subseteq (A)^+ \rightarrow$ No, $A^+ = ABCH$

b) Does $G \rightarrow R$? == is $R \subseteq (G)^+ \rightarrow$ No, $G^+ = G$

Maka, dapat disimpulkan bahwa AG adalah candidate key dari R.

Uses of Attribute Closure

- Testing for superkey:
 - compute closure of α
 - check if α^+ contains all attributes of R

- Testing functional dependencies
 - to check if $\alpha \rightarrow \beta$ holds, just check if $\beta \subseteq \alpha^+$
- Computing closure of F^+
 - for each $\gamma \subseteq R$ find the closure of γ^+
 - for each $S \subseteq \gamma^+$ output a functional dependency $\gamma \rightarrow S$

Canonical Cover

- The effort spent in checking for violations can be reduced by testing a simplified set of functional dependencies that has the same closure as the given set.
- The simplified set is termed the **canonical cover**
- To define a canonical cover, we must first define **extraneous attributes**
 - An attribute of a functional dependency in F is extraneous if we can remove it without changing F^+

Extraneous Attributes

Left-hand side

- Removing an attribute from the LHS of an FD could make it a stronger constraint.
- **For example:**
 removing B from $AB \rightarrow C$, gives a possibly stronger $A \rightarrow C$. It may be stronger because $A \rightarrow C$ logically implies $AB \rightarrow C$. But, $AB \rightarrow C$ doesn't logically imply $A \rightarrow C$ on its own. Instead, we may be able to remove B from $AB \rightarrow C$ safely. Suppose that $F = \{AB \rightarrow C, A \rightarrow D, D \rightarrow C\}$ then we can show that F logically implies $A \rightarrow C$. making B extraneous in $AB \rightarrow C$
- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F
- Attribute A is extraneous in α if
 - $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
- To test if attribute A is extraneous,
 - Let $\gamma = \alpha - \{A\}$.
 - Check if $\gamma \rightarrow \beta$ can be inferred from F
 - Compute γ^+ using the dependencies in F
 - If γ^+ includes all attributes in β then A is extraneous in α

• Let $F = \{AB \rightarrow C, A \rightarrow B, B \rightarrow C\}$
 • To check if B is extraneous in $AB \rightarrow C$ we check if F implies $F' = \{A \rightarrow C, A \rightarrow B, B \rightarrow C\}$
 • Check if $A \rightarrow C$ can be inferred from F

- Compute A^+ under F
- A^+ is ABC , which includes C
- This implies that B is extraneous

Right-hand side

- Removing an attribute from the RHS of an FD could make it a weaker constraint.
- **For example:** if we have $AB \rightarrow CD$ and remove C, we get the possibly weaker result $AB \rightarrow D$. It may be weaker because using just $AB \rightarrow D$, we can no longer infer $AB \rightarrow C$

But, we may be able to remove C from $AB \rightarrow CD$ safely.

Suppose that $F = \{AB \rightarrow CD, A \rightarrow C\}$.

Even after replacing $AB \rightarrow CD$ by $AB \rightarrow D$, we can still infer $AB \rightarrow C$. Thus, $AB \rightarrow CD$.

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in β if
 - $A \in \beta$ and $(F - [\alpha \rightarrow \beta]) \cup (\alpha \rightarrow (\beta - A))$ logically implies F
- To test if attribute A is extraneous:
 - Let $F' = (F - \{\alpha \rightarrow \beta\}) \cup (\alpha \rightarrow (\beta - A))$
 - Check that α^+ using F' contains A; if it does, A is extraneous in β

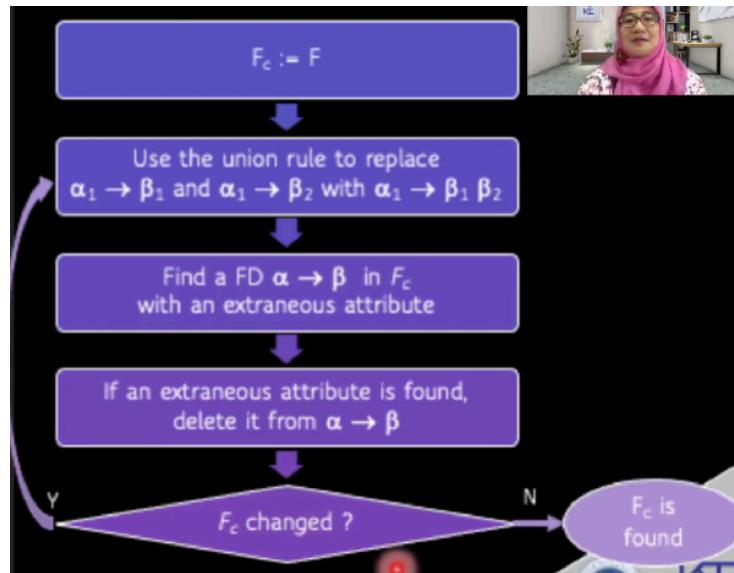
```
• Let  $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$ 
• To check if C is extraneous in  $AB \rightarrow CD$ ,
  we check if  $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$ 
  implies F
• Check if  $AB \rightarrow CD$  can be inferred from  $F'$ 
  • Compute  $AB^+$  under  $F'$ 
  •  $AB^+$  is ABCDE, which includes CD
  • This implies that C is extraneous
```

Canonical Cover

A canonical cover for F is a set of dependencies F_c such that:

- F logically implies all dependencies in F_c , and
- F_c logically implies all dependencies in F, and
- No functional dependency in F_c contains an extraneous attribute, and
- Each left side of functional dependency in F_c is unique
 - That is, there are no two dependencies in F_c
 - $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ such that
 - $\alpha_1 = \alpha_2$

Algorithm to compute a canonical cover for F:



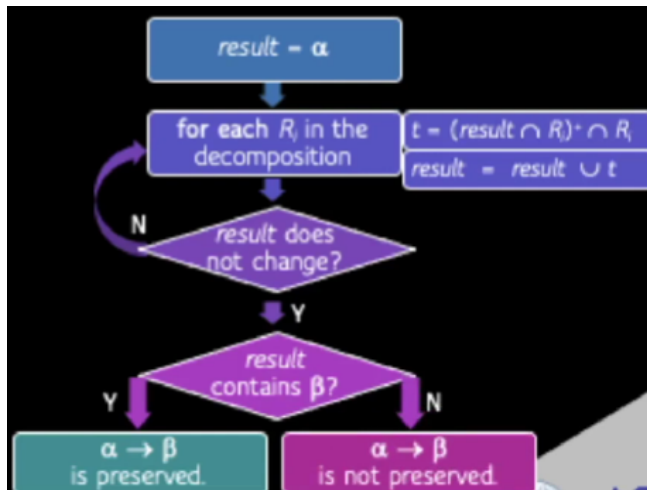
$R = (A, B, C)$
 $F = [A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C]$
 Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 • Set is now $[A \rightarrow BC, B \rightarrow C, AB \rightarrow C]$
 A is extraneous in $AB \rightarrow C$
 • Set is now $[A \rightarrow BC, B \rightarrow C]$
 C is extraneous in $A \rightarrow BC$
 • Set is now $[A \rightarrow B, B \rightarrow C]$
 The canonical cover is: $[A \rightarrow B, B \rightarrow C]$

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is dependency preserving, if $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$
- Using the above definition, testing for dependency preservation take exponential time.
- Let F be the set of dependencies on schema R and let $R_1, R_2, \dots, R_n$ be a decomposition of R
- The restriction of F to R_i is the set F_i of all functional dependencies in F^+ that include only attributes of R_i .
- The set of restrictions $F_1, F_2, \dots, F_n$ is the set of functional dependencies that can be checked efficiently

Testing for Dependency Preservation

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$, apply the following test (with attribute closure done with respect to F)

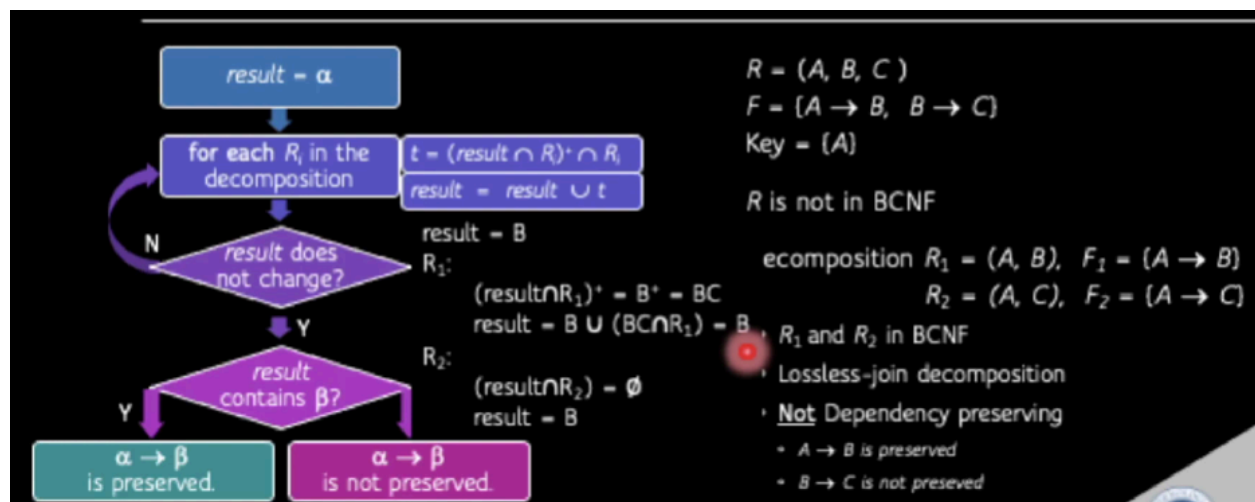


$R = (A, B, C)$
 $F = [A \rightarrow B, B \rightarrow C]$
 Key = $\{A\}$

R is not in BCNF

Decomposition $R_1 = (A, B), F_1 = [A \rightarrow B]$
 $R_2 = (B, C), F_2 = [B \rightarrow C]$

- R_1 and R_2 in BCNF
- Lossless-join decomposition
- Dependency preserving



Algorithm for Decomposition using FDs

Testing for BCNF

- To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF: verify that α is a superkey
- Simplified test: to check if R is in BCNF, it suffices to check only the dependencies in F.
- However, simplified test using only F is incorrect when testing a relation in a decomposition of R

Consider $R = (A, B, C, D, E)$ with $F = \{A \rightarrow B, BC \rightarrow D\}$

Decompose R into

$R_1 = (A, B)$ and $R_2 = (A, C, D, E)$

Neither of the dependencies in F contain only attributes from (A, C, D, E) so we might be misled into thinking R_2 satisfies BCNF

In fact, dependency $AC \rightarrow D$ in F^+ shows R_2 is in BCNF

Testing Decomposition for BCNF

To check if a relation R_i in a decomposition of R is in BCNF, either:

- test R_i for BCNF with respect to the restriction of F^+ to R_i (that is, all FDs in F^+ that contain only attributes from R_i)
- use the original set of dependencies F that hold on R, but with **additional test**
 - For every set of attributes $\alpha \subseteq R_i$, check that α^+ either includes no attribute of $R_i - \alpha$ (hanya terdapat FD trivial yang terdefinisi dengan LHS = α pada R_i), or includes all attributes of R_i (terdefinisi $\alpha \rightarrow R_i$ dan α superkey pada R_i).

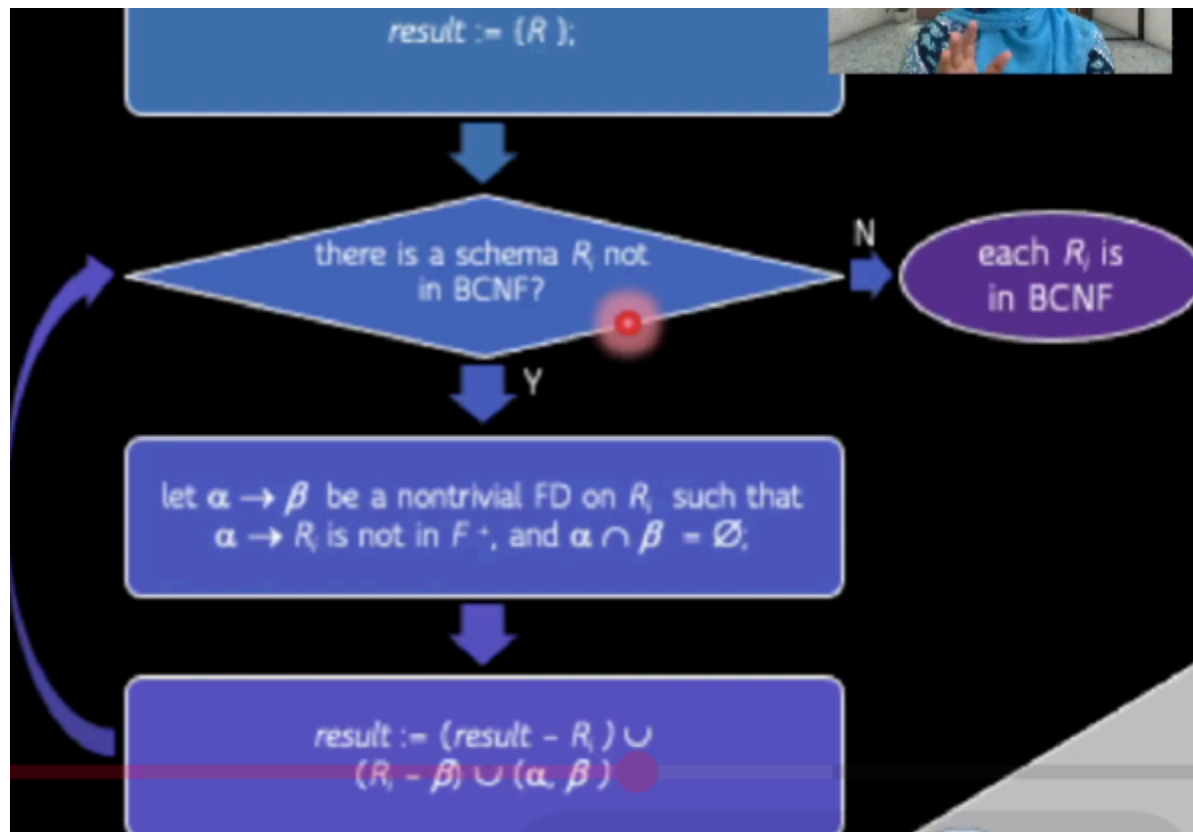
untuk kedua kasus ini, R_i adalah relasi yang BCNF relatif terhadap FD dengan LHS α

- Apabila persyaratan dilanggar, artinya pada R_i terdefinisi FD $\alpha \rightarrow \beta$ dengan β merupakan atribut" pada α^+ yang bukan elemen dari α tapi merupakan

bagian dari skema R_i . Dengan demikian, α bukan superkey pada R_i sehingga menyebabkan pelanggaran aturan BCNF.

- We use above dependency to decompose R_i .

BCNF Decomposition Algorithm



class (*course_id*, *title*, *dept_name*, *credits*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)

Functional dependencies:

- $course_id \rightarrow title, dept_name, credits$
- $building, room_number \rightarrow capacity$
- $course_id, sec_id, semester, year \rightarrow building, room_number, time_slot_id$

A candidate key (*course_id*, *sec_id*, *semester*, *year*)

BCNF Decomposition:

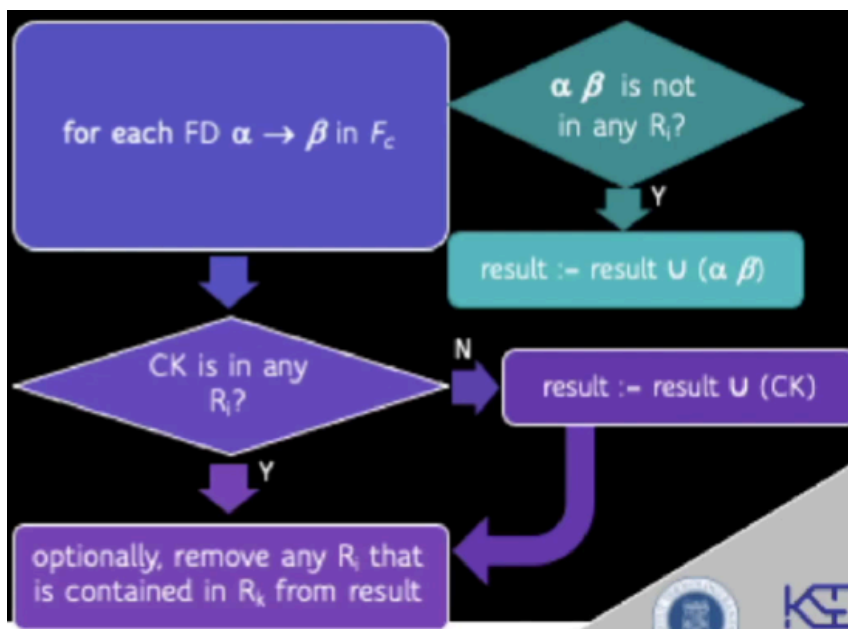
- $course_id \rightarrow title, dept_name, credits$ holds
 - but $course_id$ is not a superkey.
- We replace *class* by:
 - *course* (*course_id*, *title*, *dept_name*, *credits*) **BCNF**
 - *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)
- $building, room_number \rightarrow capacity$ holds on *class-1*
 - but $\{building, room_number\}$ is not a superkey for *class-1*.
- We replace *class-1* by:
 - *classroom* (*building*, *room_number*, *capacity*) **BCNF**
 - *section* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*) **BCNF**

Testing for 3NF

- Need to check **only** FDs in **F**, need not check all FDs in F^+
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
- This test is rather more expensive, since it involve finding candidate keys

3NF Decomposition Algorithm

- Let F_c be a canonical cover for F that constraints R.



• Relation schema:
 $\text{cust_banker_branch} = (\text{customer_id}, \text{employee_id}, \text{branch_name}, \text{type})$

• The functional dependencies for this relation schema are:

1. $\text{customer_id}, \text{employee_id} \rightarrow \text{branch_name}, \text{type}$
2. $\text{employee_id} \rightarrow \text{branch_name}$
3. $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

• We first compute a canonical cover

- branch_name is extraneous in the r.h.s. of the 1st dependency
- No other attribute is extraneous, so we get $F_c =$
 $\text{customer_id}, \text{employee_id} \rightarrow \text{type}$
 $\text{employee_id} \rightarrow \text{branch_name}$
 $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

• The for loop generates following 3NF schema:

- $(\text{customer_id}, \text{employee_id}, \text{type})$
- $(\text{employee_id}, \text{branch_name})$
- $(\text{customer_id}, \text{branch_name}, \text{employee_id})$

• Observe that $(\text{customer_id}, \text{employee_id}, \text{type})$ contains a candidate key of the original schema, so no further relation schema needs be added

• Detect and delete schemas, such as $(\text{employee_id}, \text{branch_name})$, which are subsets of other schemas

- result will not depend on the order in which FDs are considered

• The resultant simplified 3NF schema is:

- $(\text{customer_id}, \text{employee_id}, \text{type})$
- $(\text{customer_id}, \text{branch_name}, \text{employee_id})$